Indian Institute of Technology Madras

CS6886: Systems Engineering for Deep Learning

Course Project Report

Exploring LLS: Granularity, Early Exit and Scaling LLS

Yukash Ram
CS23B069

Hayagriv Venkatesh
ED23B024

October 2026

Abstract

LLS [1], a local learning rule inspired by neural activity synchronization, trains each layer with its own local loss against a fixed periodic basis, removing end-to-end backpropagation (BP) and most of its training memory at a cost in accuracy. We study four questions. For *granularity*, we group layers into blocks that share one LLS head and find that where the cuts go matters more than how many heads remain: merging the early layers closes 89% of the LLS–BP accuracy gap on VGG8 at 1.29× the per-layer LLS memory, and 94% on MobileNetV1 at 2.56× (BP needs 8.21×), while simple uniform or pooling-aligned rules do not transfer between models. For *early exit*, we use the LLS heads as free exits: within one point of the full network, MobileNetV1 runs at 10–15% of its compute and VGG8 at 57–68%, a gap set by the architecture rather than the stopping rule. Coarser partitions give better exits, fixed heads match learned linear exits except where the head input is very wide, and BP with jointly trained exits remains better at every cost. The per-layer heads are also interpretable: they are calibrated, they show where a network stops improving, and their agreement tracks how hard an input is. For *scalability*, grouping ResNet-20 into three resolution-aligned blocks improves on per-block LLS-MxM by 15.8 points, and a two-block Wide-ResNet-20 is 2.5 points below BP with 12.7% less memory. For *attention*, a six-block ViT-Tiny is 3.1 points below BP with 74% less memory.

Contents

1 Introduction **2**
- 1.1 Local Learning Rule . . . 2
- 1.2 Local Learning Synchronization . . . 2
- 1.3 This Report . . . 2

2 Granularity **3**
- 2.1 Setup . . . 3
- 2.2 VGG8 . . . 5
- 2.3 MobileNetV1 . . . 7
- 2.4 Per-Layer Analysis . . . 9
- 2.5 Partition Search . . . 9
- 2.6 Conclusion . . . 10

3 Early Exit with LLS Heads **10**
- 3.1 Setup . . . 11
- 3.2 Compute Savings . . . 12
- 3.3 Stopping Rules . . . 12
- 3.4 Granularity and Exits . . . 13
- 3.5 Fixed vs Learned Exits . . . 14
- 3.6 Exit Quality and Difficulty . . . 14
- 3.7 Conclusion . . . 15

4 Scalability: ResNet **16**
- 4.1 Setup . . . 16
- 4.2 Results . . . 17
- 4.3 Conclusion . . . 17

5 Attention: ViT **17**
- 5.1 Setup . . . 18
- 5.2 Results . . . 18
- 5.3 Conclusion . . . 18

6 Conclusion **19**

1 Introduction

1.1 Local Learning Rule

Backpropagation (BP) computes each layer's update from an error propagated back through every layer above it. This *update-locks* the network (a layer cannot update until the full forward and backward passes reach it) and forces all intermediate activations to be stored until the backward pass, so training memory grows as $O(Ln)$ for L layers of n neurons. Both make BP costly for on-device training. A *local learning rule* instead updates layer l from its own input $h^{(l-1)}$, output $h^{(l)}$ and a locally generated learning signal $\delta^{(l)}$, so layers train independently and need not keep each other's activations. The main families differ in how $\delta^{(l)}$ is produced:

- **Random feedback**: a fixed random matrix $B^{(l)}$ carries the signal, either the output error (feedback alignment [10], DFA [11], $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} B^{(l)}$) or the label (DRTP [6], $\delta^{(l)} = y^* B^{(l)}$).
- **Forward-only**: a second forward pass on negative data replaces the backward pass, and each layer maximizes a local "goodness" (Forward-Forward [8]).
- **Local classifiers**: each layer or block gets a trainable auxiliary classifier and is trained by BP inside the block only [12, 3]; these scale to deep CNNs but add parameters and compute.

1.2 Local Learning Synchronization

LLS [1] is a local learning rule inspired by *neural activity synchronization*: in the brain, neurons that respond to the same stimulus tend to fire together in a common rhythm. LLS imitates this by asking the activity of each layer to follow a periodic pattern whose frequency identifies the class, so the class is encoded in when neurons fire rather than in a learned output layer. In effect it replaces the trainable local classifier with a fixed one. Layer l, with output length T_l (after average pooling for convolutional layers), gets a fixed basis $B^{(l)} \in \mathbb{R}^{C \times T_l}$ whose row c is a periodic wave $b_c^{(l)} = g(f_c, t)$, $t = 1, \ldots, T_l$, with one frequency f_c per class (we use a square wave, as the paper does). The layer is trained with a cross-entropy on its projection onto the basis,

$$\mathcal{L}^{(l)} = \mathrm{CE}\left(h^{(l)} B^{(l)\top},\, y^*\right), \qquad \delta^{(l)} = \left(p^{(l)} - y^*\right) B^{(l)}, \tag{1}$$

where $p^{(l)}$ is the softmax of the projection. Since waves of different frequencies are nearly orthogonal, the basis acts as a fixed classifier that separates the classes without being trained. The learning signal $\delta^{(l)}$ depends only on the layer's output, the label and the fixed basis, so no error crosses layers, each layer updates as soon as its forward pass is done, and training memory is $O(n_{\max})$, set by the largest layer. The basis has no parameters and can be generated on the fly. The paper reports accuracy close to BP on CIFAR10/100, Imagenette and TinyImageNet with up to 300× fewer MACs and about half the memory.

We call a layer's projection onto $B^{(l)}$ its *LLS head*. The paper also proposes two variants with a few learned parameters: LLS-M learns one scale per class, and LLS-MxM learns a $C \times C$ matrix that mixes the fixed class scores; we use LLS-MxM in Sections 4 and 5. Section 2 generalizes LLS from one head per layer to one head per block of layers.

1.3 This Report

Per-layer LLS trails BP by several points. This report studies how far that gap can be closed and what else the local heads can be used for, through four questions from our proposal:

1. **Granularity** (Section 2): can a block of layers share one LLS head, and which grouping gives the best accuracy for a given training memory?
2. **Early exit** (Section 3): can the LLS heads, which every block already has, be used to stop inference early, and do they tell us anything interpretable about the network?
3. **Scalability** (Section 4): does LLS scale beyond VGG8 and MobileNetV1 to deeper residual networks?
4. **Attention** (Section 5): does LLS work on self-attention, i.e. on a Vision Transformer?

Granularity and early exit use VGG8 and MobileNetV1 on CIFAR10 (and CIFAR100); scalability uses ResNet-20 and Wide-ResNet-20, and attention ViT-Tiny, on CIFAR10. Code, run registry and wandb logs are in our repository.

2 Granularity

Per-layer LLS and BP are two ends of one axis. In between, the network can be cut into *blocks* of consecutive layers, trained with BP inside each block and with one LLS head at its end. We explored this axis to answer three questions: (1) what happens to accuracy and training memory as the number of LLS heads changes, (2) which cuts give the best accuracy for a given memory budget, and (3) whether a simple rule for choosing the cuts carries over from one architecture to another.

2.1 Setup

Partitions. We split each network into *units*, the smallest pieces the upstream LLS code trains separately: the 7 layers of VGG8 (6 convolutions, 1 fully connected) and the 14 layers of MobileNetV1 (the first convolution and 13 depthwise-separable blocks) [9]. A *partition* groups consecutive units into blocks and is written as the list of block sizes; for example, $(2, 5)$ is a block of the first two layers followed by a block of the remaining five (Figure 1). $(1, 1, \ldots, 1)$ is per-layer LLS, and a single block (n) is one LLS head on an otherwise BP-trained network. Every gap between two units is either a cut or not, so a network with n units has 2^{n-1} partitions: 64 for VGG8 and 8,192 for MobileNetV1.

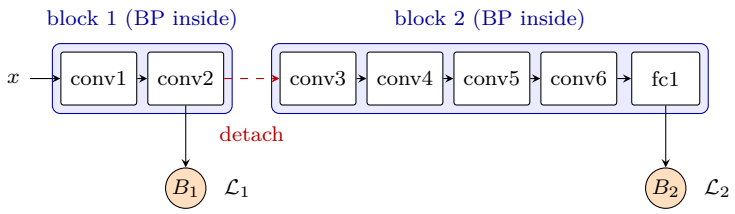


Figure 1: Partition $(2, 5)$ of VGG8. Each block is trained with BP from its own fixed-basis LLS head B_k and loss $\mathcal{L}_k$; the input to each block is detached, so no gradient crosses a block boundary. Per-layer LLS is $(1, 1, 1, 1, 1, 1, 1)$.

Partitioning schemes. We considered the following ways of choosing the cuts:

- **Uniform**: blocks of k units, the last block taking the remainder.

- **Pooling-aligned**: a cut wherever the spatial resolution changes.
- **Early or late merging**: merge only the first few or only the last few layers and keep the rest per-layer, to see where in the network merging helps.
- **Exhaustive sweep**: train every partition. This is affordable for VGG8's 64 partitions and gives the best achievable choice to compare the other schemes against.
- **Short-run search**: train every partition that fits a memory budget for only a few epochs and keep the best one (Section 2.5).

Block-wise training. We extended the upstream LLS code so that a head can sit after any unit. Each block ends in the LLS head of Section 1.2 and has its own optimizer. In a training step, block k runs forward on the *detached* output of block $k-1$, computes its local loss, backpropagates it through its own layers only and updates them (Code 1). A linear classifier on the detached output of the last block makes the final prediction. With one unit per block our code reproduces upstream LLS; we checked that the losses match step by step. The intuition is that a per-layer head forces every layer to be a good classifier on its own, while inside a block BP lets several layers cooperate on building features, and the memory grows only to that of the largest block.

```
def step(self, x: Tensor, y: Tensor) -> Out:
    h = x
    for block, opt in zip(self.blocks, self.opts):
        h = block(h.detach())                    # no gradient into earlier blocks
        loss, s = block.head(h, y)               # local LLS loss of this block
        opt.zero_grad(set_to_none=True)
        loss.backward()                          # BP inside this block only
        opt.step()
    logits = self.cls(h.detach())                # final linear classifier
    ...
```

Code 1: One training step of a partitioned network (`GranularLLS.step`).

Training memory. Because block k finishes its backward pass (and frees its activations) before block $k+1$ runs forward, only one block's activations are alive at a time. Let M_k be the activation memory of block k: the bytes of all tensors that autograd saves during the forward pass of block k and its head, which must be kept for the backward pass. The training memory of a partition with blocks $1, \ldots, K$ is then set by its largest block, while BP keeps the activations of every layer ℓ of the network at once:

$$M_{\text{partition}} = \max_{k=1,\ldots,K} M_k, \qquad M_{\text{BP}} = \sum_{\ell=1}^{n} M_\ell. \tag{2}$$

We measure M_k exactly rather than estimating it, by running the block once at batch size 128 and recording every tensor autograd saves, excluding weights and buffers (Code 2). Weights and optimizer state are the same for every partition and are not counted. We report memory relative to per-layer LLS, i.e. $M_{\text{partition}}/M_{\text{LLS}}$.

```python
def saved_bytes(fn: Callable[[], Tensor], static: Iterable[Tensor]) -> int:
    skip = {t.untyped_storage().data_ptr() for t in static}       # weights, buffers
    seen: dict[int, int] = {}
    def pack(t: Tensor) -> Tensor:
        s = t.untyped_storage()
        if s.data_ptr() not in skip:
            seen[s.data_ptr()] = s.nbytes()                       # count each storage once
        return t
    with torch.autograd.graph.saved_tensors_hooks(pack, lambda t: t):
        out = fn()                                                # e.g. head(block(x), y)[0]
    return sum(seen.values())
```

Code 2: Measuring M_k: bytes autograd keeps for the backward pass.

Gap closed. Every partition lies between per-layer LLS and BP, so we also express its test accuracy a as the share of the LLS–BP accuracy gap it recovers:

$$G = \frac{a - a_{\mathrm{LLS}}}{a_{\mathrm{BP}} - a_{\mathrm{LLS}}}, \tag{3}$$

where a_{LLS} and a_{BP} are the test accuracies of per-layer LLS and BP with the same training budget. $G = 0\%$ means no better than per-layer LLS and $G = 100\%$ means BP accuracy.

Pareto front. A partition is *Pareto-optimal* if no other partition is both at least as accurate and uses no more memory (and strictly better in one of the two). The *Pareto front* is the set of these partitions: for any memory budget, the best partition is on it, and every point below it is wasting memory or accuracy.

Training setup. We train on CIFAR10 (and CIFAR100) with random crop and flip, batch size 128, and schedule-free AdamW [4] at learning rate 5×10^{-3} with no weight decay, as in the LLS code. We hold out a stratified 5k validation split and report test accuracy at the best validation epoch. All 106 runs are logged to Weights & Biases, and every figure and table here is regenerated from those logs by one script.

2.2 VGG8

As a sanity check, our 300-epoch per-layer LLS reaches 86.6% and BP 93.5% on CIFAR10, close to the published 88.6% and 94.1%. We then trained all 64 VGG8 partitions for 60 epochs (Figure 2). Points form columns because memory depends only on the largest block; within a column, the choice of cuts moves accuracy by 2–3 points.

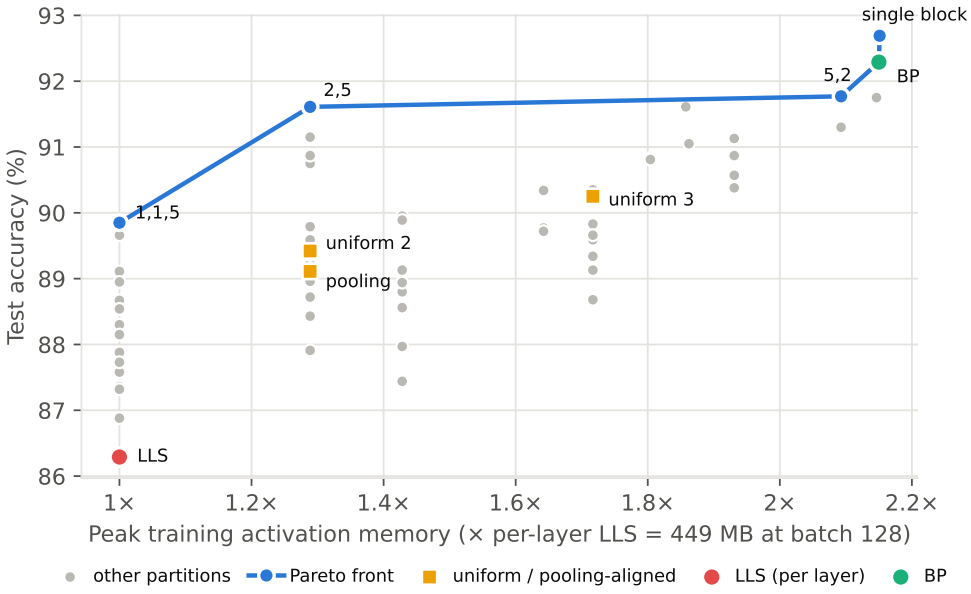


Figure 2: All 64 VGG8 partitions on CIFAR10 (60 epochs, seed 0): test accuracy against peak training activation memory relative to per-layer LLS. Blue: Pareto front. Yellow squares: uniform and pooling-aligned partitions.

The simple schemes do not work well: uniform blocks of two (89.42%) and the pooling-aligned cuts (89.11%) sit 2.2–2.5 points below the best partition with the same memory, and uniform blocks of three are below the front as well. The front itself (Table 1) has three notable points:

- $(1, 1, 5)$ uses exactly the per-layer LLS memory and closes 60% of the gap. The last five layers have small activations, so merging them does not raise the peak in Eq. (2): this gain is free.
- $(2, 5)$ is the knee: $G = 89\%$ at $1.29\times$ LLS memory. Merging only the first two layers buys most of the remaining accuracy.
- Going from $(2, 5)$ to BP costs $2.15\times$ LLS memory for the last 0.7 points.

Table 1: Pareto front of the VGG8 sweep (60 epochs, seed 0). G (Eq. 3) uses the 3-seed LLS mean (86.20%) and the BP run at 60 epochs (92.29%).

Partition	Test acc. (%)	G	Memory (MB)	× LLS	Train MMAC
LLS (1,1,1,1,1,1,1)	86.20	0%	448.5	1.00	1446
(1,1,5)	89.85	60%	448.5	1.00	1712
(2,5)	91.61	89%	578.1	1.29	2014
(5,2)	91.77	91%	938.7	2.09	2127
BP	92.29	100%	964.4	2.15	2165
Single block (7)	92.69	107%	964.9	2.15	2165

To check that these differences are not seed noise, we repeated the key partitions with three seeds (Table 2). The standard deviation is 0.1–0.3 points, against 0.9–3.7 points between consecutive front points.

Table 2: VGG8 on CIFAR10, 60 epochs, seeds 0–2.

Partition	Mean test acc. (%)	Std
LLS (1,1,1,1,1,1,1,1)	86.20	0.32
(1,1,5)	89.93	0.19
(2,5)	91.44	0.19
BP	92.30	0.11
Single block (7)	92.50	0.23

CIFAR100. We trained four of these partitions on CIFAR100 for 100 epochs (Table 3). Per-layer LLS loses the most accuracy from CIFAR10 to CIFAR100 (31.8 points), while the good partitions lose about as much as BP: $(1, 1, 5)$ is 9.0 points above LLS at the same memory, more than twice its CIFAR10 gain, and $(2, 5)$ matches BP (67.14% vs 67.23%) at 1.29× LLS memory. Choosing the partition well matters more as the number of classes grows.

Table 3: VGG8 on CIFAR10 and CIFAR100. CIFAR100 is trained for 100 epochs; CIFAR10 is the best within the first 100 epochs of the 300-epoch run, or † the 60-epoch sweep.

Partition	CIFAR10 (%)	CIFAR100 (%)	Drop (points)
LLS (1,1,1,1,1,1,1,1)	86.62	54.81	31.8
Pooling-aligned (2,2,1,1,1)	89.55	59.35	30.2
(1,1,5)	89.85†	63.80	26.1
(2,5)	91.61†	67.14	24.5
BP	92.78	67.23	25.6

2.3 MobileNetV1

MobileNetV1's 8,192 partitions are too many to sweep, and its per-layer LLS reaches only 67.8%, 16 points below BP. Guided by the VGG8 front, where merging the first layers gave most of the gain, we trained 14 partitions for 300 epochs: early merges of growing size, late merges, the uniform and pooling-aligned schemes, and coarser variants of the best early merge (Table 4, Figure 3).

Table 4: MobileNetV1 on CIFAR10, 300 epochs, seed 0. 1×8 means eight blocks of one unit.

Partition	Test acc. (%)	G	Memory (MB)	× LLS
LLS (1×14)	67.82	0%	40.5	1.00
Merge last 2 (1×12, 2)	67.45	−2%	47.7	1.18
Merge last 4 (1×10, 4)	66.70	−7%	95.9	2.37
Merge first 2 (2, 1×12)	71.72	24%	50.0	1.23
Uniform $k = 2$ (2×7)	77.28	58%	62.2	1.54
Pooling-aligned (2,2,2,6,2)	77.39	59%	144.2	3.56
(3, 1×11)	78.61	67%	73.3	1.81
(4, 1×10)	81.72	86%	103.7	2.56
(4, 2, 1×8)	83.03	94%	103.7	2.56
(4, 2, 2, 2, 2, 2)	82.83	93%	103.7	2.56
(4, 2, 6, 1, 1)	82.46	90%	144.2	3.56
(4, 2, 6, 2)	83.02	94%	144.2	3.56
Single block (14)	82.74	92%	332.5	8.21
BP	84.02	100%	332.5	8.21

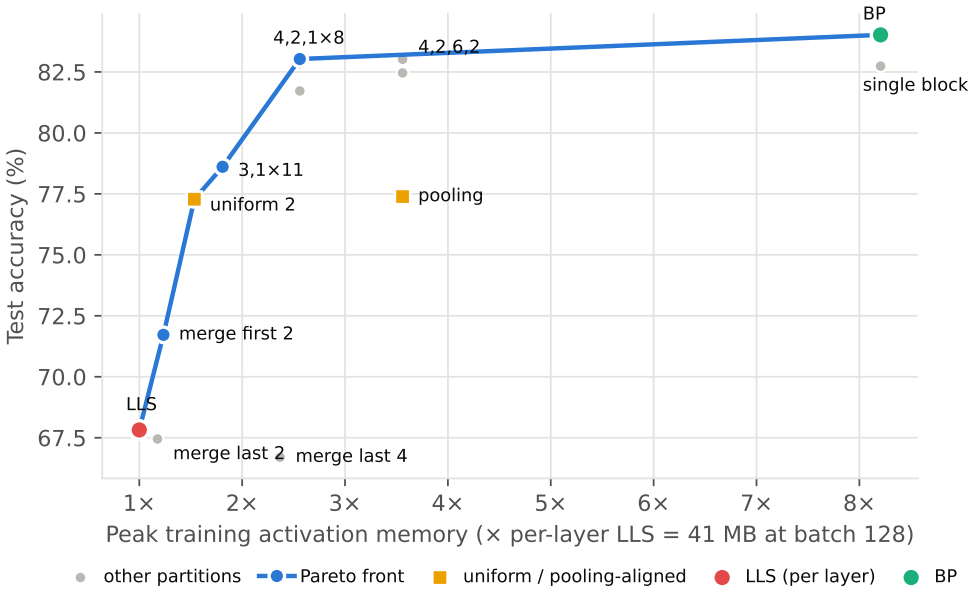


Figure 3: MobileNetV1 partitions on CIFAR10 (300 epochs): test accuracy against peak training activation memory relative to per-layer LLS (40.5 MB at batch 128).

- **Merging late layers does nothing.** Merging the last 2 or 4 layers gives 67.5% and 66.7%, no better than LLS, at up to 2.37× its memory.
- **Merging early layers helps, but more of them are needed than on VGG8.** Merging the first two gains only 3.9 points; merging the first four and then the next two, $(4, 2, 1 \times 8)$, reaches 83.0% ($G = 94\%$) at 2.56× LLS memory, while BP needs 8.21×.
- **Merging further adds nothing.** $(4, 2, 6, 2)$ reaches the same 83.0% at 3.56× memory.

- **The simple schemes stop at 77%.** Uniform blocks of two are on the front at 1.54× only because no other partition we trained uses that little memory, and reach 77.3%; pooling-aligned reaches 77.4% at 3.56×, 5.6 points below $(4, 2, 6, 2)$ at the same memory.

The memory saving is much larger than on VGG8 because MobileNetV1's activations are spread over many layers of similar size, so the sum in Eq. (2) is far larger than the maximum.

2.4 Per-Layer Analysis

To understand why early merging works on both networks, we evaluated each layer's own LLS head in the per-layer runs (Figure 4). On VGG8 head accuracy jumps from 48% at layer 1 to 80% at layer 2 and then rises slowly to 87%. On MobileNetV1 it climbs over the first six layers and then stays flat at 68–69% for the remaining eight: the second half of the network adds nothing.

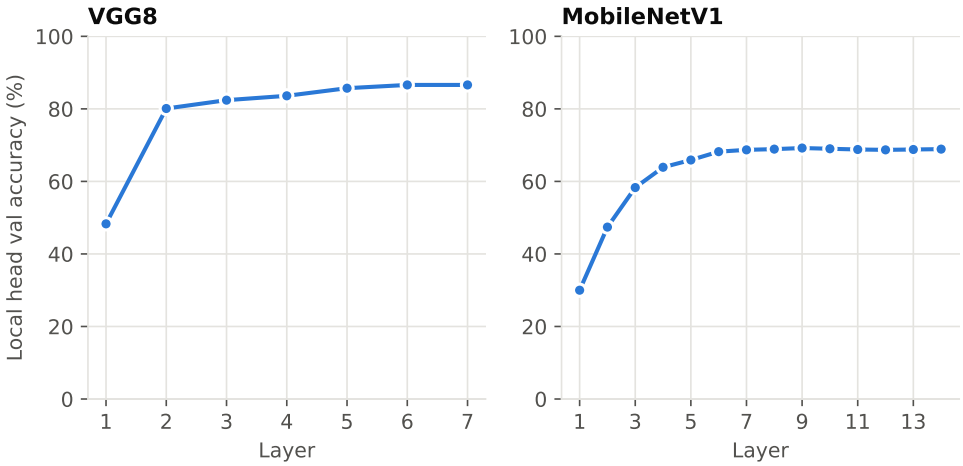


Figure 4: Validation accuracy of each layer's local LLS head in per-layer LLS (300 epochs).

Our interpretation: a local head forces its layer to make the classes separable immediately. The first layers see only small image patches, so this greedy objective throws away information that later layers would need, and the later layers cannot recover it, which is why MobileNetV1 plateaus. Merging the first layers removes their heads, so they are trained for the layers above them instead. Merging late layers cannot help, because the information is already lost by then. This explains all three observations above: $(2, 5)$ beats $(5, 2)$ per MB on VGG8, late merges do nothing on MobileNetV1, and MobileNetV1 needs a larger early block because its accuracy keeps rising over more layers (six, against two on VGG8).

2.5 Partition Search

The fixed schemes do not transfer. On VGG8, uniform and pooling-aligned cuts are 2.2–2.5 points below the best partition with the same memory; on MobileNetV1 they stop at 77%, 6 points short of the best early merge. The good early block is also model-dependent: two layers on VGG8, four plus two on MobileNetV1. So the partition has to be searched per model.

A cheaper search is to train every candidate for only a few epochs and keep the best. On VGG8, picking the partition with the best validation accuracy after 10 of the 60 epochs finds the true best

in 4 of 5 memory budgets and loses 1.3 points in the fifth ($(3,4)$ instead of $(2,5)$), at one sixth of the cost of the full sweep (Table 5).

Table 5: Choosing a partition per memory budget on VGG8. "Full": best validation accuracy over 60 epochs; "10-epoch": best within the first 10 epochs. Test accuracy is from the 60-epoch run of the chosen partition.

Budget (MB)	× LLS	Full	Test (%)	10-epoch	Test (%)
482	1.07	(1,1,5)	89.85	(1,1,5)	89.85
579	1.29	(2,5)	91.61	(2,5)	91.61
675	1.50	(2,5)	91.61	(2,5)	91.61
771	1.72	(2,5)	91.61	(3,4)	90.35
964	2.15	(6,1)	91.75	(6,1)	91.75

Even short runs still train every candidate, which does not scale to MobileNetV1's 8,192 partitions. Searching the cuts directly is what SEDONA [13] proposes: it learns where to split a network into locally trained blocks with a differentiable architecture search, which is exactly the per-model choice our results call for. We did not attempt it within the project's time, and applying such a search to LLS blocks is the natural extension of this section.

2.6 Conclusion

The number of LLS heads can be varied freely from one per layer to one per network, and removing heads raises accuracy, but where the cuts go matters more than how many there are. Merging the early layers recovers most of the gap to BP at little memory cost: $(2,5)$ closes 89% of the gap at 1.29× the per-layer LLS memory on VGG8, and $(4,2,1\times 8)$ closes 94% at 2.56× on MobileNetV1, where BP needs 8.21×. Merging late layers does not help, and merging small late layers is free ($(1,1,5)$ gains 3.7 points at exactly the LLS memory). The gap matters more with more classes: on CIFAR100 per-layer LLS loses 31.8 points while $(2,5)$ matches BP.

The per-layer heads also explain these results. Their accuracies show that greedy early heads force the first layers to separate classes too soon, so information is lost early and later layers cannot recover it; on MobileNetV1 the last eight layers add nothing. Reading the heads of a single per-layer run therefore tells us where to merge. The right cut is still model-dependent, two layers on VGG8 but four plus two on MobileNetV1, and the simple uniform and pooling-aligned rules fall 2–6 points short. The partition has to be searched per model: short 10-epoch runs are a cheap proxy for small networks, and a learned search such as SEDONA is the way to scale it.

Limitations. The sweep uses 60 epochs and one seed (seed noise about ±0.3 points); the 300-epoch runs are single-seed; only 14 of 8,192 MobileNetV1 partitions were trained. Memory counts activations only. Two upstream choices affect all partitions equally: CIFAR100 is normalized with CIFAR10 statistics, and MobileNetV1's depthwise convolutions use padding equal to the stride, so its feature maps shrink faster than in the standard model.

3 Early Exit with LLS Heads

Every LLS block already ends in a classifier, its fixed-basis head, which is discarded after training. We explored whether these heads can serve as *free* early exits: a sample stops at the first block

whose head is confident enough, with nothing extra trained. Since every head is trained to classify, a granular LLS network is already a multi-exit network, and an exit costs only one projection onto the fixed basis.

3.1 Setup

Exit models. We reused the granularity checkpoints without retraining: MobileNetV1 (8 partitions), VGG8 (6 partitions, plus all 64 from the sweep), 3 VGG8 seeds and 4 VGG8 CIFAR100 runs. A partition with K blocks has $K+1$ exits: one per block head plus the final classifier. We stored every exit's scores on the validation and test sets once and evaluated all stopping rules offline on them.

Stopping rules. Let p_l be the softmax of exit l (with a per-exit temperature fitted on validation) and $p_l^{(1)} \geq p_l^{(2)}$ its two largest entries. A sample stops at the first exit where a signal s_l reaches a threshold τ:

$$\begin{aligned} \text{confidence:}\quad & s_l = p_l^{(1)}, \qquad \text{entropy:}\quad s_l = -H(p_l), \qquad \text{margin:}\quad s_l = p_l^{(1)} - p_l^{(2)}, \\ \text{patience:}\quad & s_l = \text{number of consecutive exits ending at } l \text{ with the same top class}, \\ \text{sync:}\quad & s_l = \frac{2}{w(w-1)} \sum_{l-w<i<j\leq l} \cos(p_i, p_j). \end{aligned} \tag{4}$$

Here $\cos(p_i, p_j) = p_i^\top p_j / (\|p_i\| \, \|p_j\|)$ is the cosine similarity between the softmax distributions of exits i and j, so s_l is the mean similarity over all pairs among the last w exits and equals 1 only when they all predict the same distribution. *Sync* is our proposal, after the synchronization idea behind LLS: stop when the last w exits agree on the whole class distribution, not only the top class (Code 3). The intuition follows from how LLS trains: every layer is pushed towards the same class-specific pattern, so once consecutive heads give the same distribution, the representation has settled and deeper layers are unlikely to change the decision. Unlike confidence, which trusts a single head, sync requires agreement across depth, so one overconfident head cannot trigger an exit; and unlike patience, which only compares the top class, it also requires the heads to agree on how uncertain they are. We also tried sync+margin, running ensembles of the exits, and skipping the first few exits. Static truncation (all samples stop at the same exit) is the baseline.

```python
def sync(p: Tensor, w: int) -> Tensor:            # p: (N samples, E exits, C classes)
    u = F.normalize(p, dim=-1)
    g = u @ u.transpose(1, 2)                     # (N, E, E) cosines between exits
    out = torch.full(p.shape[:2], float("-inf"))
    for e in range(w - 1, p.shape[1]):
        blk = g[:, e - w + 1:e + 1, e - w + 1:e + 1]
        out[:, e] = (blk.sum((1, 2)) - w) / (w * (w - 1))        # mean off-diagonal cosine
    return out
```

Code 3: Sync signal (Eq. 4) for all samples and exits at once.

Cost and operating point. A sample that stops at exit l costs the backbone MACs B_l up to that exit, plus the MACs h_i of every head it checked, plus a small decision cost; we report the mean over samples as a fraction of the full network's MACs F:

$$\text{MAC fraction} = \frac{1}{F}\, \mathbb{E}_x \Big[B_{l(x)} + \sum_{i \leq l(x)} h_i + \text{decisions} \Big]. \tag{5}$$

Heads and decisions add under 1% of F. Each rule has thousands of settings (thresholds, temperature, ensembling, skipped exits). For an allowed accuracy drop δ, we pick on validation the cheapest setting within δ of the full network and report that same setting on test. We use $\delta = 1\%$ throughout.

Learned exits. For comparison we trained linear probes on the frozen LLS backbones, exits on a frozen BP backbone, and a BP network trained jointly with an exit after every layer. Each learned exit sees the same input as the LLS head at that position.

3.2 Compute Savings

At $\delta = 1\%$, MobileNetV1 runs at 10–15% of its MACs (7–10× fewer) and VGG8 only at 57–68% (Table 6). The per-exit profile (Figure 5) shows why: MobileNet's exit accuracy plateaus by block 5–6, at about 15% of the MACs, while VGG8 spends 42% of its MACs before its first useful exit (conv2, 79%), because its first convolutions run at full 32×32 resolution.

Table 6: Early exit at $\delta = 1\%$ (validation-chosen, test numbers).

	MAC fraction	speedup	test drop (pt)	best rule
MobileNetV1, 7 partitions	0.10–0.15	6.9–10.0×	0.5–1.6	margin, sync
VGG8, 5 partitions	0.57–0.68	1.5–1.7×	0.7–1.0	margin
VGG8 CIFAR100, 4 partitions	0.76–0.91	1.1–1.3×	1.2–1.6	confidence

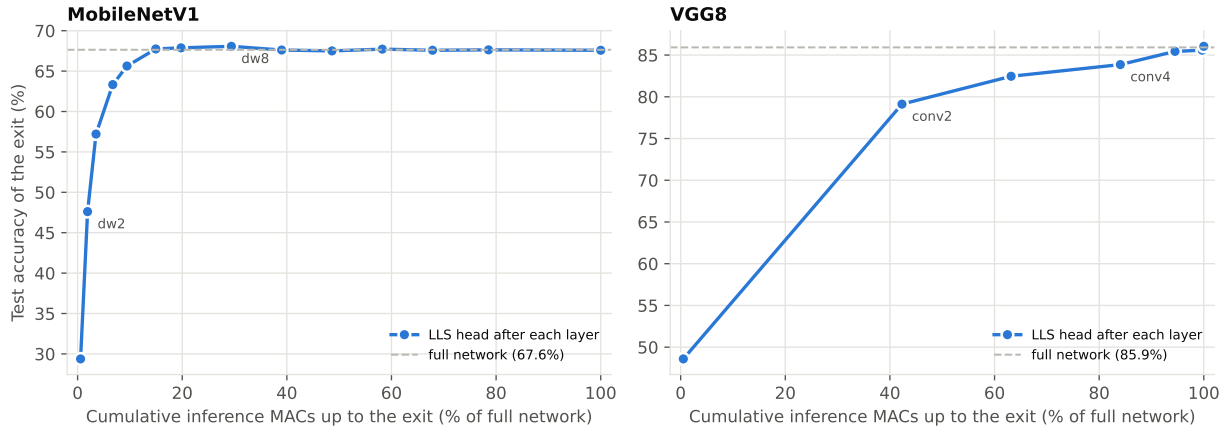


Figure 5: Per-exit test accuracy against the cumulative MACs to reach the exit, for per-layer LLS.

Inference: an exit can only save the compute after it, so the architecture, not the stopping rule, bounds the savings.

3.3 Stopping Rules

Margin is chosen most often (34 of the 64 VGG8 partitions, then confidence 20 and entropy 6); sync wins only on the finest MobileNet partitions. On VGG8 the rules separate clearly (Figure 6); on MobileNet all are close to static truncation. We expected agreement across exits to matter because

fixed-basis heads would be badly calibrated, but their calibration error (ECE) is only 0.02–0.08, and about 0.01 after temperature fitting. Ensembling the exits helps only at full depth (+0.5–1.8 points), because the weak early exits dilute the deciding one.

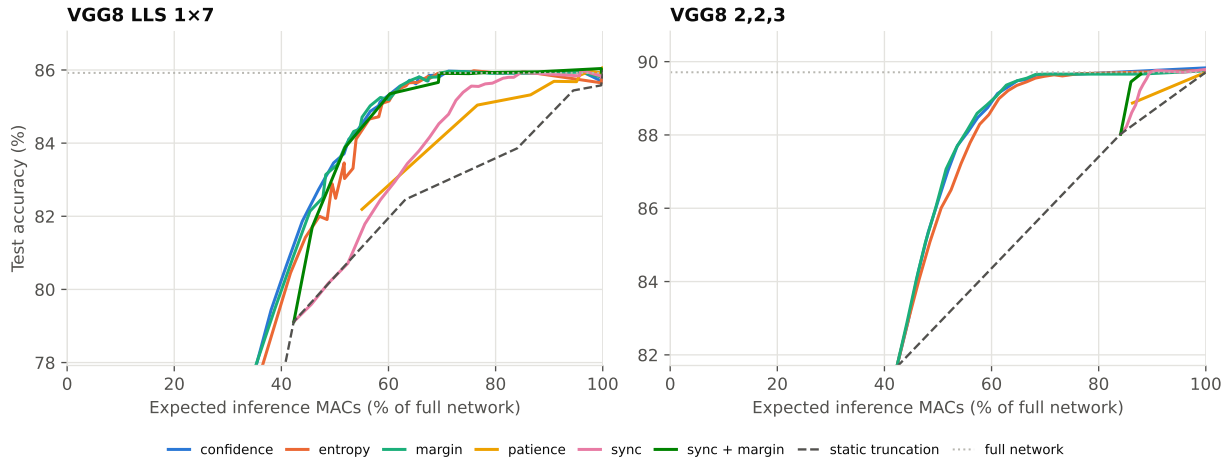


Figure 6: VGG8: accuracy against MAC fraction for each stopping rule (validation-chosen thresholds, test numbers). Dashed: static truncation; dotted: full network.

Inference: the heads are calibrated, so a single head's margin is usually enough.

3.4 Granularity and Exits

Fewer, stronger exits beat many weak ones. On MobileNet, $(4, 2, 6, 2)$ reaches 82.3% at 10% of the MACs, against 67.2% at 12% for per-layer LLS. Across the 64 VGG8 partitions (Figure 7), the cheapest start with a conv1+conv2 block (e.g. $(2, 1, 1, 1, 2)$: 87.4% at 0.55 of the MACs): merging the first two layers removes the weak conv1 exit and makes the strong conv2 exit the first one. Partitions whose first block has four or more layers save almost nothing. Across three seeds the operating points move by only ± 0.02 in MAC fraction.

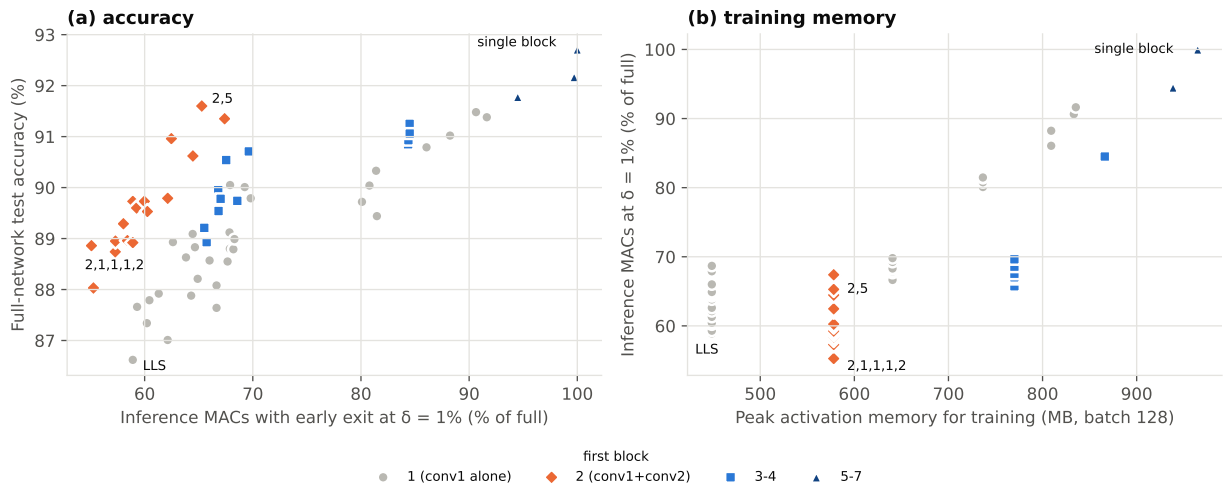


Figure 7: All 64 VGG8 partitions at $\delta = 1\%$: (a) full-network accuracy against inference MAC fraction, (b) inference MAC fraction against training memory. Color and marker give the size of the first block.

Inference: merging the first layers helps both training (Section 2.4) and exiting.

3.5 Fixed vs Learned Exits

On VGG8, linear probes match the fixed heads at every exit except conv1 (Table 7). On MobileNet they are much better at the first exits (exit 1: 58.4% vs 29.4%), which makes its operating points up to 2.5× cheaper; the later exits are unchanged. BP with jointly trained exits is still best on both networks, because the LLS networks' lower full accuracy caps their exits.

Table 7: Learned vs fixed exits at $\delta = 1\%$: test accuracy (%) @ MAC fraction.

Net	Backbone	fixed LLS heads	linear probes	BP, frozen exits	BP, joint exits
MobileNetV1	LLS 1 layer/block	67.2 @ 0.12	67.5 @ 0.07	–	–
	LLS 4,2,6,2	82.3 @ 0.10	82.2 @ 0.08	–	–
	BP	–	–	82.8 @ 0.10	**85.6 @ 0.09**
VGG8	LLS 1 layer/block	85.2 @ 0.60	85.2 @ 0.55	–	–
	LLS 3,3,1	89.2 @ 0.68	89.3 @ 0.67	–	–
	BP	–	–	92.1 @ 0.70	**91.8 @ 0.52**

Inference: fixed heads are nearly as good as learned linear exits at no training cost; LLS's advantage over BP is training memory, not exit quality.

3.6 Exit Quality and Difficulty

Weak exits. The fixed basis is not the bottleneck: its rows are nearly orthogonal (largest $|\cos|$ 0.035–0.046 on CIFAR10). Exit accuracy instead follows how well the features separate the closest pair of classes (cat–dog in 253 of 256 heads): the cosine between their mean head outputs correlates with exit accuracy at $r = -0.81$. Overthinking (correct at an earlier exit but wrong at the last) reaches 17% on the finest MobileNet partition, against 9–11% for BP exits.

Difficulty. We measured each test image's difficulty by its k-NN *prediction depth* [2]: the earliest exit from which nearest-neighbor classification of that exit's features agrees with the network's prediction at every later exit. Accuracy falls from 96% at depth 0 to 50% at depth 7 on VGG8 (Figure 8). Harder images exit later: the rank correlation (Spearman) between prediction depth and exit taken is 0.44–0.59 on VGG8 and 0.29–0.40 on MobileNet. Easy images also sync sooner, but no more than they reach a high margin.

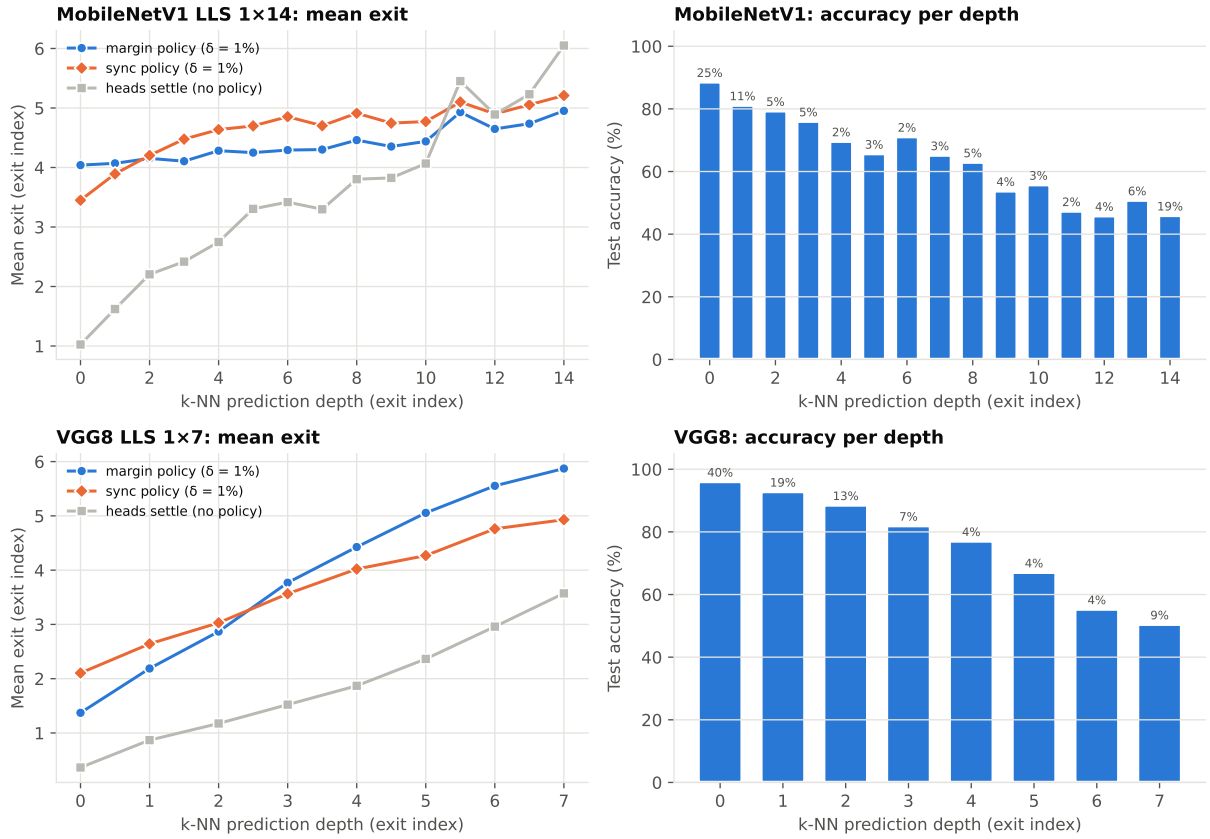


Figure 8: Per-layer LLS. Left: mean exit taken per prediction depth (margin and sync rules, $\delta = 1\%$) and the depth where the heads' predictions settle. Right: test accuracy per prediction depth (labels: share of samples).

Inference: exits are limited by class separation in the features, not by the basis, and exiting adapts to difficulty on VGG8 but is close to a fixed truncation on fine MobileNet partitions.

3.7 Conclusion

LLS heads give early exit for free: no extra training and under 1% extra MACs. How much they save is set by the architecture rather than by the stopping rule. Within one point of the full network, MobileNetV1 runs at 10–15% of its MACs, but VGG8 only at 57–68%, because VGG8 spends most of its compute before its first useful exit. A single head's margin is usually enough to decide when to stop; our sync rule, which looks for agreement across exits, helps only on the finest partitions. The partitions that train best, with the first layers merged, also give the best exits. Fixed heads are nearly as good as learned linear exits, but BP with jointly trained exits is better at every cost, so the advantage of LLS remains its training memory.

On interpretability, the proposal asked whether the per-layer LLS outputs can be read in a way that buys memory or compute efficiency. Our answer is yes. Each head is a calibrated class prediction at its depth (ECE about 0.01 after temperature fitting), so its confidence can be trusted as an exit signal, which gives the compute saving. The point at which the heads' predictions settle tracks how hard an image is (Spearman 0.42–0.64 with k-NN prediction depth), and easy classes (car, ship, frog) settle earlier than hard ones (cat, dog, bird). The heads also show where a network stops improving, which is what tells us where to merge layers in Section 2 and so gives the memory saving. Finally, the class geometry at each head shows that weak exits come from poorly separated classes, mostly cat versus dog, and not from the fixed basis, which is nearly orthogonal.

Limitations. Most runs use one seed and last-epoch weights; operating points are chosen on a 5k validation split; savings are in MACs, not measured latency.

4 Scalability: ResNet

The original LLS work evaluates only VGG and MobileNet models. We tested whether the rule, and the lesson from Section 2 that a few blocks recover most of the lost accuracy, carry over to residual networks [7].

4.1 Setup

Units. We reused the partition code of Section 2.1 and changed only what a unit is. In a ResNet, a unit is a complete basic block: both convolutions, their batch normalization and activations, the shortcut projection and the residual addition (Code 4). A block boundary therefore never cuts through a skip connection; cutting one would detach the identity path that makes deep ResNets trainable. ResNet-20 has 10 units: the stem convolution and nine basic blocks in three stages of 16, 32 and 64 channels.

```
def forward(self, x: Tensor) -> Tensor:          # BasicBlock = one unit
    identity = self.shortcut(x)                  # identity or 1x1 conv when the shape changes
    out = self.act(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    return self.act(out + identity)
```

Code 4: A ResNet unit: the shortcut stays inside the unit, so a detach between units never splits it.

Heads and models. Each block can use the fixed LLS head, LLS-M (one learned scale per class) or LLS-MxM (a learned $C \times C$ matrix that mixes the fixed class scores) [1], and the network can predict either with a detached linear classifier, as in the rest of this report, or directly with its last LLS head. We implemented ResNet-20/32/56 and a Wide-ResNet-20 with doubled channels (32/64/128).

Runs. On ResNet-20 we compared BP, per-block LLS, per-block LLS-MxM and the resolution-aligned partition $(4, 3, 3)$ with LLS-MxM, whose cuts fall at the two stage transitions. Aligned cuts follow the stages, where the feature hierarchy changes naturally. On Wide-ResNet-20 we compared BP with a two-block $(7, 3)$ LLS-MxM model that predicts with its last LLS head; the wider layers give each local block more capacity to fit its own loss. All runs use CIFAR10 with the same

augmentation, 45k/5k split, batch size 128, seed 0 and schedule-free AdamW [4] as the rest of the report, with the test accuracy reported at the best validation epoch. Memory and training MACs are analytic.

4.2 Results

Table 8 compares each LLS model with BP on the same network. On ResNet-20, per-block LLS reaches 51.4%, 36.9 points below BP, and LLS-MxM improves on it by 4.6 points. Merging the blocks into three resolution-aligned groups helps much more: $(4, 3, 3)$ reaches 71.9%, which improves on per-block LLS-MxM by 15.8 points and uses 40% less memory than BP, but is still 16.4 points below BP. Per-block LLS has the lowest memory (56.5 MB), but losing all credit assignment across blocks costs too much accuracy on a network this deep.

Table 8: Scalability on CIFAR10 (seed 0). Memory: analytic peak activation memory. The BP and per-block runs on ResNet-20 stopped early (no validation improvement for 10 epochs).

Network	Method	Epochs	Acc. (%)	Memory (MB)
ResNet-20	BP	48	88.3	281.5
	Per-block LLS	57	51.4	56.5
	Per-block LLS-MxM	57	56.1	56.5
	$(4, 3, 3)$ LLS-MxM	200	71.9	170.0
Wide-ResNet-20	BP	300	92.4	561.6
	$(7, 3)$ LLS-MxM, LLS readout	300	89.8	490.0

The wider network closes most of the gap. With matched 300-epoch training, $(7, 3)$ reaches 89.8%, 2.5 points below BP, with 12.7% less memory than BP (490.0 vs 561.6 MB) and 3% fewer training MACs (471.6 vs 486.2 MMAC). It is not faster in wall-clock time (22.9 vs 22.2 s per epoch): the analytic saving is small and is offset by the extra heads and optimizers.

4.3 Conclusion

LLS scales to residual networks if the units respect the skip connections and are grouped into a few blocks. As on VGG8 and MobileNetV1, per-block LLS loses too much accuracy and coarser blocks recover most of it, and the wider network, whose blocks have more capacity, comes within 2.5 points of BP. The memory saving shrinks as the blocks get larger, from 80% for per-block LLS to 13% for the two-block Wide-ResNet, so on ResNets the accuracy–memory trade-off is steeper than on VGG8.

Limitations. All runs use one seed. The ResNet-20 BP and per-block runs stopped early (48 and 57 epochs) while $(4, 3, 3)$ trained for 200, so only the Wide-ResNet comparison is matched in training length. Memory and MACs are analytic, and the measured time per epoch shows no speedup.

5 Attention: ViT

We also tested whether LLS works on self-attention, using a Vision Transformer [5].

5.1 Setup

Model and units. We implemented a CIFAR ViT-Tiny with 12 pre-norm encoder blocks, 192-dimensional tokens, three attention heads, an MLP ratio of 4 and 4×4 patches. A unit is one encoder block with both of its residual branches (attention and MLP), so, as with the ResNets, no skip connection is cut. The first unit also holds the patch embedding, CLS token and positional embeddings, and the last one the final layer normalization.

LLS head on tokens. An encoder block outputs a sequence of tokens rather than a feature map, so the head first reduces it to one vector: it concatenates the CLS token with the mean of the patch tokens (Code 5). The CLS token only gathers class information in later blocks, while the mean patch token gives a stable summary from the first block on, so the local loss supervises both.

```
if h.dim() == 3:                                    # (batch, tokens, dim)
    h = torch.cat((h[:, 0], h[:, 1:].mean(dim=1)), dim=1)   # CLS token + mean patch token
    h = F.adaptive_avg_pool1d(h, size)
```

Code 5: Reducing token activations for the LLS head (inside `pool_flat`).

Runs. We trained two matched models for 100 epochs on the same V100 GPU: BP, and a six-block $(2, 2, 2, 2, 2, 2)$ partition with LLS-MxM heads and a detached linear classifier. Training, model selection and data are as in Section 4.1; memory here is the measured peak GPU memory.

5.2 Results

Both models reached their best validation accuracy at epoch 96 (Table 9). The six-block LLS-MxM model reaches 77.9%, 3.1 points below BP, with 74% less GPU memory than BP (353.3 vs 1368.2 MB), but its epochs take 19% longer (23.3 vs 19.6 s).

Table 9: Attention on CIFAR10 (ViT-Tiny, 100 epochs, seed 0). Memory: measured peak GPU memory.

Method	Acc. (%)	Memory (MB)	Time (s/epoch)
BP	81.0	1368.2	19.6
$(2, 2, 2, 2, 2, 2)$ LLS-MxM	77.9	353.3	23.3

5.3 Conclusion

LLS transfers to self-attention with a small accuracy cost and the largest memory saving in this project. Attention stores large intermediate tensors in every block, so keeping only two blocks' activations at a time removes most of the training memory, while each two-block group still has enough depth to fit its local loss. The extra heads and optimizers make each epoch slower, so, as for the CNNs, the benefit is memory rather than speed.

Limitations. One seed and one partition; we did not compare other block sizes, the fixed LLS head, or CIFAR100 on the ViT.

6 Conclusion

Granularity and early exit are two views of the same structure: the blocks of a partition set both the training memory and the exits available at inference. Merging the first layers is the common lesson. It recovers most of the accuracy lost by per-layer LLS at a small memory cost, and it turns the first exit into a useful one. The best partition is model-dependent, so it has to be searched rather than fixed by a rule.

The per-layer outputs of LLS are also interpretable in a useful way. Each head is a calibrated prediction at its depth; the heads show where a network stops improving, which guides the partition and hence the training memory; and their agreement tracks how hard an input is, which lets easy inputs exit early and saves compute. LLS heads, however, do not beat BP with jointly trained exits, so the case for LLS rests on its lower training memory.

The same lesson holds beyond VGG8 and MobileNetV1. On ResNets, units must keep the skip connections whole, and merging them into a few blocks recovers most of the accuracy lost by per-block LLS, bringing Wide-ResNet-20 within 2.5 points of BP. On ViT-Tiny, six two-block groups come within 3.1 points of BP with 74% less memory, the largest saving in this project. Across all models, LLS trades a few points of accuracy for training memory, not for speed.

References

[1] Marco Paul E. Apolinario, Arani Roy, and Kaushik Roy. LLS: Local learning rule for deep neural networks inspired by neural activity synchronization. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025.

[2] Robert J. N. Baldock, Hartmut Maennel, and Behnam Neyshabur. Deep learning through the lens of example difficulty. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[3] Eugene Belilovsky, Michael Eickenberg, and Edouard Oyallon. Decoupled greedy learning of CNNs. In *International Conference on Machine Learning (ICML)*, 2020.

[4] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. *arXiv preprint arXiv:2405.15682*, 2024.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[6] Charlotte Frenkel, Martin Lefebvre, and David Bol. Learning without feedback: Fixed random learning signals allow for feedforward training of deep neural networks. *Frontiers in Neuroscience*, 15:629892, 2021.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

[9] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[10] Timothy P. Lillicrap, Daniel Cownden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016.

[11] Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[12] Arild Nøkland and Lars Hiller Eidnes. Training neural networks with local error signals. In *International Conference on Machine Learning (ICML)*, 2019.

[13] Myeongjang Pyeon, Jihwan Moon, Taeyoung Hahn, and Gunhee Kim. SEDONA: Search for decoupled neural networks toward greedy block-wise learning. In *International Conference on Learning Representations (ICLR)*, 2021.